

Shashwat — Applied AI/ML Engineering

Project Portfolio, DSA & System Design Prep Roadmap

1. Where To Focus: Career Direction

Given a strong base in Python and classic ML (regression/classification, Scikit-learn), backend fundamentals (Node/Express, REST APIs, MySQL/MongoDB), and early exposure to MLOps (MLflow, Docker, Flask), **Applied AI/ML Engineering** is the right lane — not a deep-research ML Scientist track. This role is about building and shipping AI-powered systems (RAG, agents, ML services) with solid software engineering around them, which matches this profile far better than heavy theoretical ML.

Two changes that will move the needle fastest:

- **Standardize on Python for all AI portfolio work.** Keep Node/Express as a "can also do backend" line on the resume, but every AI project should be Python end-to-end (FastAPI + LangGraph + ML tooling) since that is the ecosystem AI Engineer interviews and take-homes are built around.
- **Move from "trained a model" to "shipped a system."** The existing ML projects (Hotel Booking, CDSS) prove modeling ability. What's missing for an AI Engineer resume is: agent orchestration (LangGraph), production-style serving (FastAPI), observability (tracing/metrics), and evaluation (automated, repeatable — not just accuracy on a test set).

The five projects below are chosen specifically to cover those gaps. They're ordered by suggested build sequence — each one reuses skills from the last, so the workload compounds instead of restarting each time.

2. Project Portfolio (6 Projects)

Note on the "Outcome" lines below: brackets like [X]% are placeholders. Once he builds and evaluates the project, he should replace them with the actual numbers his eval pipeline produces — never estimate or guess a metric for a resume.

PROJECT 1 DocuMind — Production RAG API with Automated Evaluation

Ask questions over your own documents, with citations, and a CI pipeline that proves the answers are actually good.

Problem: Most student RAG projects stop at "it answers questions." This one adds what production RAG actually needs: retrieval tuning backed by measurable evaluation, and full tracing of every query.

Tech Stack: FastAPI, LangChain (or LlamaIndex), Qdrant or ChromaDB (vector store), sentence-transformers embeddings (free/local — no API cost), Redis (query cache), Ragas (RAG evaluation: faithfulness, answer relevancy, context precision/recall), Langfuse (tracing/observability), Docker, GitHub Actions (CI)

Architecture / Build Plan:

- Ingestion pipeline: PDF/text loader → chunking (experiment with fixed-size vs. semantic chunking) → embeddings → vector store
- FastAPI endpoints: /ingest, /query, /feedback (thumbs up/down for future fine-tuning of retrieval)
- Retrieval step: similarity search + a lightweight reranker (cross-encoder) before passing context to the LLM
- Wrap every pipeline stage (retrieve, rerank, generate) in Langfuse trace spans so latency and token cost per stage are visible on a dashboard

- Build a ~30–50 question "golden set" with expected answers; run Ragas metrics against it automatically on every code push via GitHub Actions — this is what turns "I built RAG" into "I built a RAG system with regression-tested quality"

Resume Bullets (fill in real numbers after building):

- Built a production-style RAG API (FastAPI + LangChain + Qdrant) serving document Q&A with source citations, achieving [X]% faithfulness and [X]% context precision on a 50-question golden evaluation set (Ragas)
- Instrumented the full retrieval-generation pipeline with Langfuse tracing, reducing p95 query latency from [X]s to [X]s by adding a Redis cache layer and tuning chunk size
- Set up a CI pipeline (GitHub Actions) that runs the evaluation suite on every commit, catching retrieval-quality regressions before deployment

Level It Up (Stretch Goals):

- Add hybrid search (BM25 keyword search + vector search, combined) — pure vector search misses exact-match terms like product codes or names, and this is a very common production fix
- Try query rewriting or HyDE (Hypothetical Document Embeddings) for vague user questions, and A/B it against plain retrieval using the same Ragas golden set — data-driven proof it helped, not just a claim
- Add multi-tenant document isolation (so User A's documents never leak into User B's answers) — a detail interviewers specifically probe for in RAG system design questions
- Load-test the API with Locust to report a concrete "handles N concurrent users at Xms p95 latency" number

PROJECT 2

AgentOps — Multi-Agent Research Assistant (LangGraph)

A planner → researcher → writer → critic agent graph that researches a topic and writes a sourced summary.

Problem: Demonstrates agentic orchestration, not just a single LLM call — the core skill LangGraph-based roles screen for.

Tech Stack: FastAPI (with Server-Sent Events for streaming), LangGraph, **MCP (Model Context Protocol)** as the tool-calling layer instead of hardcoded LangChain tool functions, OpenTelemetry + Langfuse (per-node tracing), DeepEval (LLM-as-judge evaluation), Docker

Architecture / Build Plan:

- LangGraph state machine with distinct nodes: **Planner** (breaks the task into steps) → **Researcher** (calls search/tool nodes) → **Writer** (drafts the answer) → **Critic** (checks the draft against the sources, sends it back to Writer if unsupported claims are found)
- Wire the Researcher node up as an **MCP client** connecting to one or more MCP servers (a web-search server, a calculator, a sandboxed code-execution server) instead of writing bespoke tool-call glue code — this is the direction tool integration is standardizing on industry-wide, so it's worth doing this way rather than the older "custom LangChain @tool function" pattern
- Guardrails: max iteration/loop limit, input validation on the FastAPI layer, a fallback node if a tool call fails
- Stream intermediate agent "thoughts"/step updates to the client over SSE from the FastAPI endpoint
- Instrument each LangGraph node with an OpenTelemetry span, feeding a Langfuse dashboard showing cost and latency per node (this is the single most "applied AI engineer" signal you can put on a resume)
- Build a small set of 15–20 research tasks as a regression test suite; score plan quality, tool-call correctness, and final-answer correctness with DeepEval's LLM-as-judge metrics, run in CI

Resume Bullets:

- Designed a multi-agent research assistant using LangGraph (planner/researcher/writer/critic nodes) with MCP-based tool integration and a self-correction loop, served via a streaming FastAPI endpoint
- Added per-node observability (OpenTelemetry + Langfuse), cutting average token cost per query by [X]% after identifying a redundant tool-call pattern in the Researcher node
- Built an automated regression test suite (DeepEval, [N] tasks) to catch agent-quality regressions before they reach production

Level It Up (Stretch Goals):

- Add long-term memory: store past research sessions in a vector store so the agent can recall "we already looked into this last week" instead of starting cold every time
- Add model routing — use a small/cheap model for the Planner node and a stronger model only for the Writer node, then report the cost savings versus using one model everywhere
- Add a hard token/cost budget per session with a kill-switch node, since runaway agent loops are the #1 real-world complaint about agentic systems in production
- Add a human-approval gate before any tool call that has a real-world side effect (e.g. sending an email), demonstrating you understand agent autonomy needs limits

PROJECT 3 ML Model Serving & Monitoring Platform

Turns the existing Hotel Booking / CDSS models into a properly monitored, versioned production service.

Problem: Bridges his existing classic-ML projects into modern MLOps practice — versioning, monitoring, and drift detection are what separate "I trained a model" from "I can run ML in production."

Tech Stack: FastAPI, MLflow (model registry + experiment tracking — already has exposure here), Prometheus + Grafana (metrics/dashboards), Evidently AI (data/prediction drift detection), Docker Compose, GitHub Actions (CI/CD)

Architecture / Build Plan:

- Register the Hotel Booking (or CDSS) model versions in the MLflow Model Registry instead of a flat pickle file; FastAPI loads the "Production"-stage model at startup and exposes `/predict` and `/model-info`
- Expose Prometheus metrics from FastAPI: request count, latency histogram, and the distribution of predicted classes/scores
- Grafana dashboard on top of Prometheus showing live traffic and prediction distribution
- Evidently AI job (scheduled or triggered) comparing incoming request data against the training distribution to flag data drift; on drift, log an alert (simple webhook/log line is enough — no need for a full alerting stack)
- Docker Compose file wiring together the API, Prometheus, Grafana, and MLflow so the whole stack runs with one command; GitHub Actions runs tests + rebuilds the image on push

Resume Bullets:

- Productionized an existing ML classifier (87%+ accuracy) with MLflow model registry versioning and a FastAPI serving layer, replacing a flat-file deployment workflow
- Built a Prometheus/Grafana monitoring stack tracking request latency and prediction drift, with Evidently AI flagging distribution shift in incoming data
- Containerized the full stack (API + monitoring + model registry) with Docker Compose for one-command local deployment

Level It Up (Stretch Goals):

- Add shadow/canary deployment: route a small % of live traffic to a new model version, log both models' predictions, and compare before fully switching over — a real MLOps pattern, not just "deploy the new model"
- Wire the drift alert to an automated retraining job (even a simple scheduled script counts) so drift detection actually does something instead of just logging
- Add a model explainability endpoint (SHAP values) so predictions come with "why," not just a number
- Load-test the serving endpoint and report throughput/latency at different replica counts, showing you thought about scaling, not just correctness

PROJECT 4 PromptBench — LLM Evaluation & Regression Testing Framework

"Unit tests" for prompts: catches quality regressions whenever a prompt or model changes.

Problem: Directly targets the "evals" gap — a dedicated evaluation tool is a strong, distinctive resume line since most student projects skip evaluation entirely.

Tech Stack: FastAPI (trigger/serve eval runs), Ragas + DeepEval (metrics: correctness, relevancy, hallucination, toxicity), SQLite/Postgres (store run history), Streamlit (results dashboard), Ollama (to test against a local open-source model alongside a hosted API model), GitHub Actions

Architecture / Build Plan:

- Define a test-case format (YAML/JSON): prompt template + input variables + expected behavior/reference answer
- FastAPI endpoint `/run-eval` executes a test suite against one or more models (e.g., a hosted API model vs. a local Ollama model) and scores each response with Ragas/DeepEval metrics
- Persist every run (prompt version, model, scores, timestamp) so quality can be tracked over time, not just as a single snapshot
- Streamlit dashboard to browse runs and see score trends across prompt versions (line chart: score vs. prompt version)
- Wire it into CI: whenever a prompt template file changes, GitHub Actions runs the eval suite and fails the build if any metric drops below a threshold — genuine "prompt regression testing"

Resume Bullets:

- Built an LLM evaluation framework (Ragas + DeepEval) that scores model outputs on faithfulness, relevancy, and hallucination, tracking quality trends across [N] prompt versions and 2 models
- Integrated automated prompt-regression testing into CI, blocking deployment when eval scores dropped below a set threshold
- Compared a hosted LLM against a local Ollama-served open-source model on cost, latency, and quality trade-offs across [N] test cases

Level It Up (Stretch Goals):

- Validate the LLM-as-judge itself: have a human manually score a sample of the same responses and report agreement (inter-rater reliability) between the human and the judge model — this is the detail that separates a real eval engineer from someone who just called an eval library
- Extend from single-turn to multi-turn conversation evals, which is meaningfully harder and closer to real agent behavior
- Add a handful of adversarial/red-team test cases (prompt injection, jailbreak attempts) to the suite, since safety evals are increasingly expected alongside quality evals
- Pipe failing eval runs into a Slack webhook so the CI integration feels like a real alerting system, not just a red X on GitHub

An agent that triages and drafts responses to support tickets, but escalates to a human when unsure.

Problem: Shows the thing hiring managers actually worry about with agents in production: knowing when *not* to trust the model, and routing that case to a person.

Tech Stack: FastAPI, LangGraph, Postgres (ticket/state storage), Redis + RQ or Celery (escalation queue), Ragas (response-quality eval), a simple PII-redaction step (regex or Presidio), Langfuse, Docker

Architecture / Build Plan:

- LangGraph flow: **Classify** ticket intent/urgency → **Retrieve** relevant KB articles (reuses the RAG pipeline from Project 1) → **Draft** a response → **Confidence check** node scores the draft (grounded-in-source? within policy?) → auto-send if confidence is high, else push to a human-review queue
- Redis + RQ (or Celery) queue for tickets that need human review, with a minimal review UI (a Streamlit page listing pending tickets is enough — no need for a full frontend build)
- PII redaction pass on incoming tickets before they're sent to the LLM or logged anywhere
- Ragas-based evaluation of auto-sent responses on a held-out ticket set, plus a manual audit sample to sanity-check the confidence threshold is calibrated correctly
- Full observability via Langfuse: what fraction of tickets got auto-resolved vs. escalated, and why

Resume Bullets:

- Built a human-in-the-loop support triage agent (LangGraph) combining RAG-based response drafting with a confidence-gated auto-send/escalate decision, reducing manual review load by [X]% in testing while keeping error-prone cases in front of a human
- Implemented a PII-redaction and confidence-scoring layer so only high-confidence, policy-compliant responses were auto-sent
- Added end-to-end observability (Langfuse) exposing auto-resolution rate and escalation reasons as a live dashboard

Level It Up (Stretch Goals):

- Feed escalated (human-corrected) tickets back into the golden eval set — a lightweight version of the "active learning" loop real support-AI teams use to keep improving over time
- Swap the hand-rolled confidence/PII checks for an actual guardrails framework (Guardrails AI or NeMo Guardrails) to show you can use the standard tooling, not just reimplement it
- Add SLA-aware prioritization so urgent tickets jump the human-review queue instead of first-in-first-out
- Add a weekly auto-generated summary report (volume, auto-resolution rate, top escalation reasons) — a small feature that signals you're thinking about the people who'd actually use this system, not just the model

A small, fast-to-build project that's still rare enough on student resumes to stand out: build an MCP server, not just consume one.

Problem: Most people only ever use someone else's MCP server. Building your own — one that exposes real tools/data — is a concrete, easy-to-demo way to show you understand the protocol, not just the buzzword.

Tech Stack: Python MCP SDK, FastAPI (if exposing it over HTTP/SSE transport instead of stdio), a real backing data source (e.g. his own GitHub activity via the GitHub API, a small SQLite database of notes, or the Hotel Booking model from Project 3), Docker

Architecture / Build Plan:

- Pick one real data source he already has: GitHub repos/issues, a personal notes database, or the ML model from Project 3
- Expose it as MCP **tools** (e.g. `search_notes`, `get_repo_issues`, `predict_booking`) and MCP **resources** where relevant, following the MCP spec's schema for tool descriptions so any MCP-compatible client can discover and call them correctly
- Test it by connecting it to Claude Desktop (or another MCP client) directly — a genuinely satisfying "it just works" demo
- Then connect the *same* server as a tool provider for the Project 2 LangGraph agent, showing the server is reusable across different clients, which is the whole point of the protocol
- Add basic auth/scoping on the server so it doesn't expose more than intended, and log every tool call for auditability

Resume Bullets:

- Built and published a custom MCP (Model Context Protocol) server exposing [GitHub / notes / model] data as callable tools, consumed by both Claude Desktop and a custom LangGraph agent
- Designed MCP tool schemas following the protocol spec, with scoped auth and per-call logging for auditability

Level It Up (Stretch Goals):

- Support both transports (stdio for local clients, HTTP/SSE for remote) instead of just one, and document the trade-off
- Replace a simple API key with proper OAuth-based auth and per-client rate limiting — the difference between a toy server and one you'd trust with real data
- Write a small test suite that checks multiple LLMs actually call your tools correctly given only the schema — tool-schema quality is a real, underrated skill
- Package it as a pip-installable module with a clean README so someone else could actually install and run it in five minutes

How these fit together on the resume: Projects 1 & 5 share the RAG/KB layer, Projects 2, 5 & 6 share tool-use and MCP, and Projects 3 & 4 both showcase evaluation/observability discipline — one for classic ML, one for LLMs. Even building 3 of the 6 (recommend starting with Project 1, then 2, then 4) is enough to credibly claim FastAPI + LangGraph + MCP + observability + evals on a resume. Project 6 is the cheapest to build (a weekend, not weeks) and is worth doing even if time is short.

3. Trending AI Engineering Concepts Worth Knowing

Beyond what's built into the projects above, these are the concepts coming up most often in AI Engineer interviews and job descriptions right now. He doesn't need to build all of them — but should be able to explain each one in a sentence or two, and Project 6 already gives him hands-on MCP experience to speak from.

MCP (Model Context Protocol) — An open standard (introduced by Anthropic) for connecting AI models/agents to external tools and data sources through a single common interface, instead of every app writing its own custom tool-calling glue code. Project 6 above is built specifically to give him real, demoable experience with it.

Agent2Agent (A2A) protocol — A complementary emerging standard for how separate agents (possibly built by different teams or vendors) discover each other and communicate/hand off tasks. Worth knowing conceptually — MCP is agent-to-tool, A2A is agent-to-agent.

Guardrails / safety layers — Frameworks like Guardrails AI, NeMo Guardrails, or Llama Guard that validate LLM inputs/outputs against rules (PII, toxicity, jailbreak attempts, schema conformance) before they reach a user. Project 5's confidence-check and PII-redaction nodes are a hand-rolled version of this pattern.

Structured outputs / function calling — Getting an LLM to reliably return JSON matching a schema (via provider-native structured-output modes or libraries like Pydantic/Instructor) instead of parsing free text — this is what makes tool calling and agent hand-offs reliable enough to put in production.

Semantic / prompt caching — Caching LLM responses by the *meaning* of a query (embedding similarity), not just exact string match, plus provider-level prompt caching for repeated system prompts/context — both are direct cost and latency levers, and a natural extension of the Redis caching already used in Project 1.

Context engineering & agent memory — The practice of deliberately managing what goes into an LLM's context window — retrieved documents, tool outputs, conversation history, short-term vs. long-term agent memory — rather than just "stuffing everything in." This is the more accurate, current framing of what used to be called "prompt engineering."

Lightweight fine-tuning (LoRA / QLoRA) — Parameter-efficient fine-tuning techniques that adapt a base model cheaply on limited hardware. Not required for an applied-engineering track, but worth knowing exists as an alternative to prompting when accuracy or cost demands it.

Suggestion: pick 2–3 of these (MCP is the easy one, given Project 6) and be ready to talk through them in interviews with a concrete example, rather than trying to namedrop all of them.

4. DSA Preparation Plan

With 200+ problems solved and a CodeChef rating already, the priority now is **pattern fluency at the medium level** and timed practice, not re-learning basics. Suggested topic checklist, roughly in order of interview frequency:

Topic	Focus
Arrays & Two Pointers	Sliding window, prefix sums, in-place partitioning
Hashing	Frequency maps, grouping, O(1) lookup tricks (two-sum family)
Binary Search	On sorted arrays <i>and</i> "binary search on the answer" problems
Strings	Pattern matching, palindrome checks, sliding window on strings
Recursion & Backtracking	Subsets, permutations, N-Queens style constraint search
Linked Lists	Reversal, cycle detection (fast/slow pointers), merging
Stacks & Queues	Monotonic stack (next greater element), sliding-window max
Trees	Traversals, BST operations, LCA, diameter/height problems
Graphs	BFS/DFS, topological sort, Union-Find, Dijkstra/shortest path
Heaps / Priority Queue	Top-K problems, merge-K-lists, median-finding
Dynamic Programming	1D & 2D DP, knapsack family, DP on strings (LCS/edit distance)
Greedy	Interval scheduling, activity selection

Practice plan: Work through the NeetCode 150 or LeetCode "Blind 75" list topic-by-topic, then switch to timed, random-topic mock sessions (e.g., on Pramp or with a friend) 3–4 weeks before interviews to simulate real conditions.

5. System Design Prep

4a. Core Fundamentals (needed regardless of AI focus):

- Scalability basics: vertical vs. horizontal scaling, load balancing
- Caching strategies (Redis), cache invalidation trade-offs
- SQL vs. NoSQL trade-offs, database indexing, sharding basics
- Message queues (Kafka/RabbitMQ/Redis Streams) for async workloads
- REST API design: pagination, rate limiting, idempotency
- CAP theorem and consistency vs. availability trade-offs at a conceptual level

4b. AI/ML-Specific System Design (the differentiator for Applied AI/ML roles):

- Designing a RAG system at scale — chunking strategy, vector index choice (HNSW vs. IVF), hybrid (keyword + vector) search
- Designing an LLM inference/serving layer — batching requests, streaming responses, GPU vs. CPU trade-offs, prompt/response caching for cost control
- Designing a recommendation system — candidate generation vs. ranking stage, cold-start handling
- Feature store design — keeping online (serving-time) and offline (training-time) features consistent

- Model monitoring & drift detection design — what to log, what triggers a retrain
- Designing an agentic system with guardrails — tool-use safety, loop limits, human-in-the-loop escalation (directly reusable from Projects 2 & 5 above)
- A/B testing design for a model rollout — how to measure whether a new model version is actually better
- Batch vs. streaming data pipeline design — where Airflow/Kafka fit

Good sources: *"Designing Machine Learning Systems"* (Chip Huyen) for the ML-specific side, and standard system design primers (e.g. the *"system-design-primer"* GitHub repo) for the general fundamentals.

6. Suggested 12-Week Roadmap

Weeks	Focus
1–2	Project 1 (DocuMind RAG API) + DSA: Arrays, Hashing, Two Pointers, Binary Search
3–4	Project 2 (AgentOps LangGraph agent, incl. MCP tool client) + DSA: Recursion/Backtracking, Linked Lists, Stacks/Queues
5	Project 6 (Personal MCP Server, weekend build) + Project 4 (PromptBench evals)
6–7	Project 3 (ML Serving & Monitoring) + DSA: Trees, Graphs
8	Resume polish + start applying + System Design fundamentals (5a)
9–10	Project 5 (SupportGraph, optional stretch) + DSA: DP, Heaps, Greedy
11–12	AI/ML System Design deep-dive (5b) + timed mock interviews (DSA + system design)

This is an aggressive but doable pace if he can dedicate consistent evenings/weekends to it. If it needs to stretch to 16–18 weeks around coursework, keep the order the same — sequencing matters more than speed.